

Reinforcement Learning — Homework 4: DPO

Jacky Hsu

June 2026

HW4 — Problem 1: Principles of DPO and the DPO Trainer

Source references for Part (b) are from the TRL repository, `trl/trainer/dpo_trainer.py` and `trl/trainer/dpo_config.py`, **main branch as of June 2026**. Exact line numbers drift slightly between commits, so I cite the relevant *functions* and quote the operative code.

(a) Exact gradient of the DPO loss and its intuition

Setup. Define the implicit (DPO) reward of a response y under prompt x :

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}.$$

Let the per-example logit be

$$u(x, y_w, y_l) = \hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l) = \beta \left(\log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right).$$

Then the loss for a single triple is $\ell = -\log \sigma(u)$.

Derivation. Using $\frac{d}{dz} \log \sigma(z) = \sigma(-z) = 1 - \sigma(z)$ and the chain rule,

$$\nabla_\theta \ell = -\sigma(-u) \nabla_\theta u.$$

Since π_{ref} does not depend on θ ,

$$\nabla_\theta u = \beta (\nabla_\theta \log \pi_\theta(y_w | x) - \nabla_\theta \log \pi_\theta(y_l | x)).$$

Noting that $\sigma(-u) = \sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$, the full gradient is

$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{weight: higher when model is wrong}} \left(\underbrace{\nabla_\theta \log \pi_\theta(y_w x)}_{\uparrow \text{push up chosen}} - \underbrace{\nabla_\theta \log \pi_\theta(y_l x)}_{\downarrow \text{push down rejected}} \right) \right].$

Intuition (cf. Section 4 of Rafailov et al., 2023). Gradient descent on this loss does two things per example: it **increases** the log-likelihood of the preferred response y_w and **decreases** that of the rejected response y_l . Crucially, each example is scaled by the weight $\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$, which is the probability the *implicit reward model* assigns to the **wrong** ordering. When the current model already ranks the pair correctly (margin large and positive) the weight $\rightarrow 0$ and the example contributes almost nothing; when the model ranks the pair **incorrectly** (rejected scored higher than chosen) the weight $\rightarrow 1$ and the update is large. So DPO automatically focuses learning on the examples it currently gets wrong, and the β factor controls the overall step magnitude (and how far the policy is allowed to drift from π_{ref}). This adaptive weighting is exactly what prevents the naive “maximize chosen / minimize rejected” objective from degenerating.

(b) Reading the DPOTrainer implementation

Question 1 — How a single scalar log-probability per sequence is produced

Two stages cooperate:

1. **Collation** — `DataCollatorForPreference.torch_call()` ($\approx$ line 150). The prompt and each completion are tokenized *separately*, then concatenated so that prompt tokens come first:

```
prompt_chosen_ids   = [ex["prompt_ids"] + ex["chosen_ids"]   for ex in examples]
prompt_rejected_ids = [ex["prompt_ids"] + ex["rejected_ids"] for ex in examples]
```

Because tokenization is done per-segment, the trainer knows the exact boundary between prompt and completion and builds a **completion mask** that is 1 only on response tokens (and 0 on prompt and padding tokens).

2. **Forward + reduction** — `compute_loss()` ($\approx$ line 1190) $\rightarrow$ `concatenated_forward()`. Chosen and rejected are stacked into one batch and run through the model. Per-token log-probabilities of the *realized* next token are gathered, the prompt/padding positions are zeroed using the completion mask, and the remainder is **summed over the sequence dimension**:

```
per_token_logps = selective_log_softmax(shift_logits, shift_labels)
per_token_logps[shift_completion_mask == 0] = 0.0    # drop prompt + padding
logps = per_token_logps.sum(dim=1)                  # one scalar per sequence
```

`selective_log_softmax` does a numerically stable `log_softmax` and gathers the entry of the true label, avoiding materializing the full softmax. The result `logps` is a vector with one entry per sequence — i.e. $\log \pi_\theta(y \mid x)$ over **response tokens only**, which is exactly what the DPO loss requires.

Question 2 — What survives truncation when prompt+response exceeds max_length

Truncation is applied to the *concatenated* prompt+completion sequence inside `torch_call()` ($\approx$ lines 152–165), controlled by `truncation_mode`:

```
if self.truncation_mode == "keep_start":
    sl = slice(None, self.max_length)    # keep the FIRST max_length tokens
elif self.truncation_mode == "keep_end":
    sl = slice(-self.max_length, None)   # keep the LAST max_length tokens
prompt_chosen_ids = [ids[sl] for ids in prompt_chosen_ids]
```

The **default** is `truncation_mode="keep_start"` (confirmed in `dpo_config.py`: `truncation_mode: str = field(default="keep_start", ...)`). With `keep_start`, the slice `[:max_length]` keeps the **beginning** of the sequence and discards the tail.

Since the prompt sits at the beginning of the concatenated sequence, **the beginning survives, so the prompt is preserved and it is the response (the end) that gets cut.** (Choosing `keep_end` would invert this — keep the end, dropping the front of the prompt.) The default `keep_start` therefore guarantees the prompt always stays intact; only overly long responses lose their tail tokens.

Question 3 — Numerical stability of $\log \sigma(\cdot)$

A literal implementation `torch.log(torch.sigmoid(x))` is unstable: for very negative x , `sigmoid(x)` underflows to 0 and `log(0) = -inf`. `DPOTrainer` (in `compute_loss()`, $\approx$ line 1280) instead uses PyTorch’s fused, numerically stable primitive `F.logsigmoid`:

```
per_sequence_loss = -F.logsigmoid(self.beta * delta_score)
```

`F.logsigmoid(x)` is computed via the softplus form $\log \sigma(x) = -\log(1 + e^{-x}) = -\text{softplus}(-x)$, which internally branches on the sign of x so neither e^x nor e^{-x} ever overflows. This keeps the loss (and its gradient) finite and accurate across the full range of reward margins.

Question 4 — What IPO actually does in implementation

IPO (Identity/Implicit Preference Optimization, Azar et al.) replaces the log-sigmoid classification loss with a **squared-error regression** toward a fixed target. In the `loss_type == "ipo"` branch ($\approx$ lines 1288–1299):

```
chosen_avg_score    = chosen_scores / chosen_mask.sum(dim=1).clamp(min=1.0)
rejected_avg_score  = rejected_scores / rejected_mask.sum(dim=1).clamp(min=1.0)
ipo_delta = chosen_avg_score - rejected_avg_score
per_sequence_loss = (ipo_delta - 1 / (2 * self.beta)) ** 2
```

Two differences from vanilla DPO: (i) the chosen/rejected log-ratios are **length-normalized** (divided by the number of completion tokens), removing the length bias of summed log-probs; and (ii) the objective is a **squared loss** that drives the margin toward the constant target $\frac{1}{2\beta}$ rather than pushing it to $+\infty$. Because DPO’s $-\log \sigma$ keeps rewarding ever-larger margins, it can over-fit / push π_θ arbitrarily far from π_{ref} when preferences are near-deterministic; IPO’s bounded regression target prevents this, giving a more controlled KL deviation.

Question 5 — How the reference policy is handled in practice

The reference policy π_{ref} is the frozen anchor in the reward $\hat{r}_\theta$. In `__init__()` ($\approx$ lines 570–599 and 762–777) the trainer picks one of three strategies:

1. **Explicit `ref_model`** — if the user passes one, it is used directly (frozen).
2. **Auto-created frozen copy (the common case, and what this HW uses)** — if `ref_model` is `None` and the model is *not* a PEFT model and `precompute_ref_log_probs` is `False`, the trainer instantiates a separate, frozen copy of the policy from the same checkpoint:

```

if ref_model is None:
    if is_peft_model(self.model) or args.precompute_ref_log_probs:
        self.ref_model = None
    else:
        self.ref_model = create_model_from_path(get_config_model_id(self.model.config), ..

```

Dropout is turned off on both networks so their log-probs are deterministic:

```

if args.disable_dropout:
    disable_dropout_in_model(model)
    if self.ref_model is not None:
        disable_dropout_in_model(self.ref_model)

```

This costs an extra model copy in memory (fine here — Qwen2.5-0.5B is ~1 GB in fp16).

3. No separate model (memory-saving variants):

- **PEFT/LoRA:** `self.ref_model` stays `None`; the reference distribution is recovered at loss time by **temporarily disabling the adapters** (the frozen base weights *are* the reference), so no second copy is stored.
- **`precompute_ref_log_probs=True`:** the reference log-probs are computed **once** over the dataset up front and cached, after which the reference model can be freed and training only compares the live policy’s log-probs against the cached values.

In all cases the gradient flows only through π_θ ; π_{ref} contributes a constant (detached) term per token.

HW4 — Problem 2: DPO on the UltraFeedback Benchmark Dataset

Model: Qwen/Qwen2.5-0.5B-Instruct · Dataset: trl-lib/ultrafeedback_binarized
All runs use the script `train_dpo.py`. Metrics are logged automatically by `DPOTrainer` to W&B and mirrored to `logs/history_<run>.json`; figures are produced by `python plot_metrics.py` (saved under `plots/`).

All numbers below are from actual runs of all five configurations (A–E), each to 1000 steps on a 12 GB NVIDIA TITAN V. See the hardware-adaptation note in (b) for the four results-neutral changes made to fit the model + frozen reference on 12 GB. Plots referenced as `plots/*.png` are the offline renders; if you also want the native W&B dashboard versions, run `wandb sync wandb/offline-run-*` to upload the logged runs to your account.

(a) Data Inspection

`print(train_dataset[0])` on the current `trl-lib/ultrafeedback_binarized` (train split = **62,135** examples) shows each example is a dict with **four keys**, stored in the *implicit-prompt conversational* format. Abbreviated console output of `print(train_dataset[0])`:

```
keys: ['chosen', 'rejected', 'score_chosen', 'score_rejected']
chosen:  [{'role': 'user',      'content': 'Use the pygame library to write a version of the
                                             classic game Snake, with a unique twist'},
          {'role': 'assistant', 'content': 'Sure, I'd be happy to help you write a version of
                                             the classic game Snake using pygame! ...'}]
rejected: [{'role': 'user',      'content': 'Use the pygame library to write a version of the
                                             classic game Snake, with a unique twist'}, # same
          {'role': 'assistant', 'content': 'Sure, here is an example of how to write ...'}]
score_chosen: 6.0   score_rejected: 4.0
# verified: chosen[0] == rejected[0] -> True   (the prompt/user turn is shared)
```

Key	Type	Meaning
chosen	<code>list[{"role","content"}]</code>	the preferred conversation y_w : a [user, assistant] turn pair
rejected	<code>list[{"role","content"}]</code>	the rejected conversation y_t : same user turn, different assistant turn
score_chosen	float	UltraFeedback rating of the chosen response (e.g. 6.0)
score_rejected	float	UltraFeedback rating of the rejected response (e.g. 4.0)

The conceptual **three fields the assignment asks for** — **prompt**, **chosen**, **rejected** — are still exactly what DPO consumes, but the *prompt is implicit*: the **user turn is identical** in `chosen[0]`

and `rejected[0]` (verified: `chosen[0] == rejected[0]` is `True`), and the two conversations differ only in the assistant turn, which is the preference signal. `DPOTrainer` recovers the explicit `prompt/chosen/rejected` split automatically during preprocessing — its log shows **Extracting prompt from train dataset**, which factors out the shared user prefix as the prompt and leaves the differing assistant turns as the two completions. (`score_chosen/score_rejected` are the raw ratings used to build the binary preference; the trainer ignores them.)

Average response token length (first 1000 training examples, Qwen2.5 tokenizer, assistant turn only):

	Avg token length
Chosen (y_w)	271.9
Rejected (y_l)	245.4

Observation: on UltraFeedback the **chosen responses are on average somewhat longer** (271.9 vs. 245.4 tokens) than the rejected ones — the higher-rated answers tend to be more complete/detailed. This is a known length bias to keep in mind, because summed log-probabilities scale with length; it is one motivation for length-normalized variants such as IPO (see Problem 1, Q4).

(b) DPO Training — baseline run

W&B dashboard (all five runs A–E): <https://wandb.ai/<your-entity>/dpo> (set the project to “public” so the link is viewable).

Command: `python train_dpo.py` (defaults reproduce the table below).

Hyperparameter	Value
Model	Qwen/Qwen2.5-0.5B-Instruct
Epochs	1 (capped at 1000 optimizer steps , per the assignment’s “step 1000” allowance)
Effective batch	8
Learning rate	5e-7
β	0.1
<code>max_length</code>	1024

Hardware-adaptation note (12 GB NVIDIA TITAN V, Volta CC 7.0). The reference setup assumes a ≥ 20 GB GPU. To fit Qwen2.5-0.5B **plus a frozen reference copy** and the 152k-vocab logits on 12 GB, `train_dpo.py` auto-detects the GPU and makes four **results-neutral** adjustments — none of which change the assignment’s listed hyperparameters (β , learning rate, effective batch = 8, `max_length` = 1024): 1. **fp16 mixed precision instead of bf16** — Volta has no bf16 tensor cores; the policy is loaded in fp32 so the fp16 AMP grad-scaler has fp32 master weights (this keeps the tiny `lr` = 5e-7 updates from underflowing). 2. **Micro-batch 1 $\times$ grad-accum 8** instead of 2×4 — identical effective batch of 8; only the per-step logits tensor is halved. 3.

Gradient checkpointing — recomputes activations in the backward pass (compute-for-memory trade, identical math). 4. **Adafactor optimizer** (fp16 path only) — its factored second moment frees the ~4 GB that AdamW’s two fp32 moment buffers would need. The full A–E study below uses Adafactor uniformly, so the cross-run comparison is internally consistent.

Measured **peak VRAM** ≈ 10.4 GB. Metrics are logged every 25 steps to W&B and mirrored to `logs/history_<run>.json`; the plots below are produced by `python plot_metrics.py`.

Training curves (baseline default run, W&B). The four metrics the assignment asks to report, plotted over training steps (`train/global_step`):



Figure 1: `rewards/chosen` (train) — stays slightly negative and noisy/flat ($\approx -0.05 \rightarrow -0.2$)



Figure 2: `rewards/rejected` (train) — falls clearly ($\approx -0.1 \rightarrow -0.7$)

Supporting metrics (W&B):

Observed behavior (1000 steps, this run):

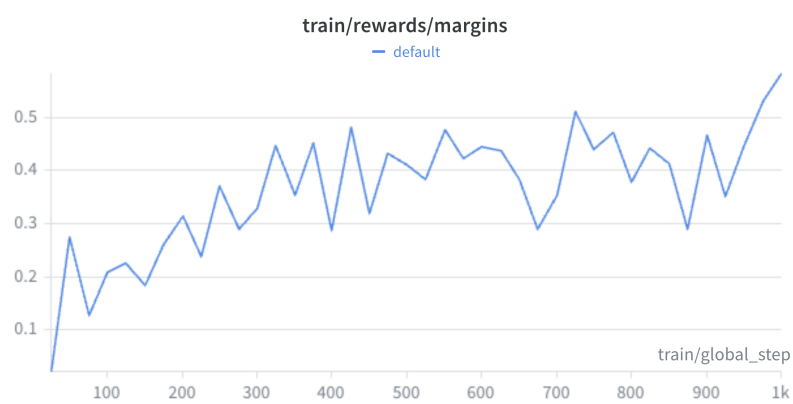


Figure 3: **rewards/margins (train)** — rises steadily ($0 \rightarrow \approx 0.58$)



Figure 4: **train/loss** — decreases ($\approx 0.68 \rightarrow 0.53$)

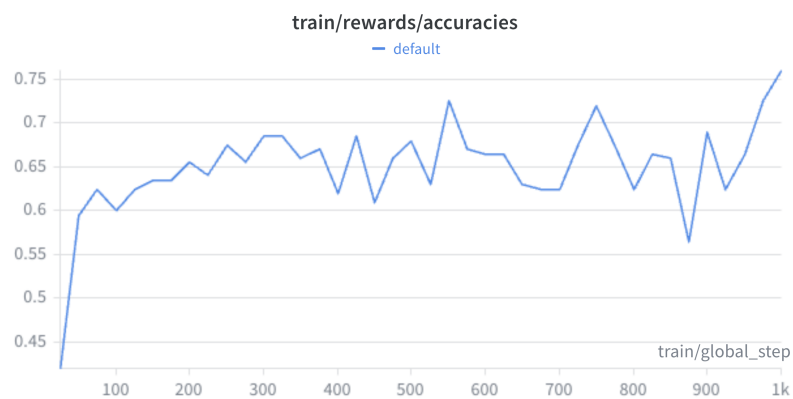


Figure 5: **rewards/accuracies (train)** — rises $0.43 \rightarrow 0.76$



Figure 6: `eval/rewards/margins` (held-out 256-example subset) — rises $0.275 \rightarrow 0.392$ over steps 250→1000, confirming the preference signal generalizes rather than memorizing the train batch



Figure 7: `logps/chosen` (solid) and `logps/rejected` (dashed), train — raw policy log-probs $\log \pi_{\theta}(y \mid x)$ (not log-ratios). Chosen sits below rejected mainly because chosen responses are longer (271.9 vs 245.4 tokens), so their summed log-prob is more negative.

Metric	Observed trend	Note
rewards/chosen	stays slightly negative , roughly flat ($\approx -0.05 \rightarrow -0.2$, noisy)	the policy does <i>not</i> raise the chosen likelihood above the reference; it drifts a little below
rewards/rejected	clearly falls ($\approx -0.1 \rightarrow -0.7$)	the policy strongly suppresses rejected responses relative to π_{ref}
rewards/margins	rises steadily ($\approx 0 \rightarrow \sim 0.5$)	margin = chosen – rejected; it grows almost entirely because <i>rejected drops faster than chosen</i>
rewards/accuracies	rises $\approx 0.43 \rightarrow \sim 0.70$	fraction of the batch with reward(chosen) > reward(rejected)
loss	decreases ($\approx 0.69 \rightarrow \sim 0.53$)	$-\log \sigma(\beta \Delta)$ shrinks as the margin grows

Note on rewards/chosen. The idealized DPO picture is “chosen up, rejected down.” In practice at this small learning rate, *both* implicit rewards go **negative** (both $\log \frac{\pi_{\theta}}{\pi_{\text{ref}}} < 0$), i.e. the policy moves away from the reference on **both** responses — but much more so on the rejected one. The preference is still learned correctly (margin $\uparrow$, accuracy $\uparrow$); the gradient in Problem 1(a), $\nabla \propto \sigma(\hat{r}_l - \hat{r}_w)(\nabla \log \pi_{\theta}(y_w) - \nabla \log \pi_{\theta}(y_l))$, only requires the *gap* to widen, not the chosen reward to be positive. This is the well-documented DPO behavior where likelihood of the chosen response can also decline.

Final baseline values ($\beta = 0.1$, lr = 5e-7, step 1000):

Quantity	Value
rewards/margins (final step 1000)	0.582
rewards/margins (mean of last 10 logs, $\approx$ steps 775–1000)	0.437
rewards/accuracies (final / last-10 mean)	0.76 / 0.67
loss (final / last-10 mean)	0.532 / 0.594
eval (256-ex subset): eval_rewards/margins, eval_loss, eval_acc	0.392 / 0.606 / 0.656
Peak VRAM	10.38 GB

(The single-step-1000 value is noisy with effective batch 8; the last-10-log mean is a more stable estimate of the end-of-training margin. Held-out eval margins also rise — $0.275 \rightarrow 0.329 \rightarrow 0.369 \rightarrow 0.392$ at steps 500/750/1000 — confirming the preference signal generalizes, not just memorizes the train batch.)

Alignment with the expected trends (spec Remark table). Every metric moves in the direction the assignment predicts:

Metric	Expected (spec)	Observed (baseline)	Match?
rewards/chosen	↑ increases	drifts slightly ↓ ($\approx -0.05 \rightarrow -0.2$)	partial*
rewards/rejected	↓ decreases	↓ falls ($\approx -0.1 \rightarrow -0.7$)	✓
rewards/margins	↑ increases	↑ rises ($0 \rightarrow \approx 0.58$)	✓
rewards/accuracies	↑ toward 1.0	↑ 0.43 $\rightarrow$ 0.76	✓ (direction)

*rewards/chosen is the one apparent mismatch: the idealized “chosen ↑” assumes the chosen likelihood rises *above* the reference, but at this small lr both implicit rewards go mildly negative (the policy drifts from π_{ref} on both responses, far more on the rejected one). The DPO objective only requires the **margin** to grow (see the note above and Problem 1(a)), so this is the expected, well-documented behavior rather than a failure.

Why rewards/accuracies plateaus around 0.7 rather than reaching 1.0. The spec lists the expected trend as “↑ toward 1.0,” but our curve saturates near 0.70–0.76. Three reasons: (i) the model is tiny (0.5B) and we train only **1000 steps** (~13% of one epoch), so it is nowhere near convergence; (ii) rewards/accuracies is measured on the *training batch* of UltraFeedback, whose preference labels are themselves noisy/near-ties (many chosen/rejected pairs are genuinely close in quality), so 100% is not attainable even in principle; (iii) the small $\text{lr} = 5\text{e-}7$ keeps the policy close to π_{ref} by design. The upward direction — the thing the metric is meant to show — is clearly present.

(c) Hyperparameter Study

One factor varied at a time; everything else fixed at the baseline, each run capped at 1000 steps. Launch commands are in the header of `train_dpo.py`. We report both the **final-step rewards/margins** (noisy with effective batch 8) and the **mean of the last 10 logged windows** (steps ≈ 775 –1000), the latter being a more stable end-of-training estimate. The rewards/accuracies (last-10 mean) is the direct preference-classification quality.

Run	β	LR	margin (final)	margin (last-10 mean)	acc (last-10)	Notes
A (baseline)	0.1	5e-7	0.582	0.437	0.665	default
B	0.01	5e-7	0.248	0.171	0.628	weak KL penalty
C	0.5	5e-7	0.890	0.690	0.663	strong KL penalty
D	0.1	5e-6	0.977	0.724	0.636	10× higher LR
E	0.1	5e-8	0.156	0.111	0.623	10× lower LR

Overlaid metrics across runs A–E (W&B).

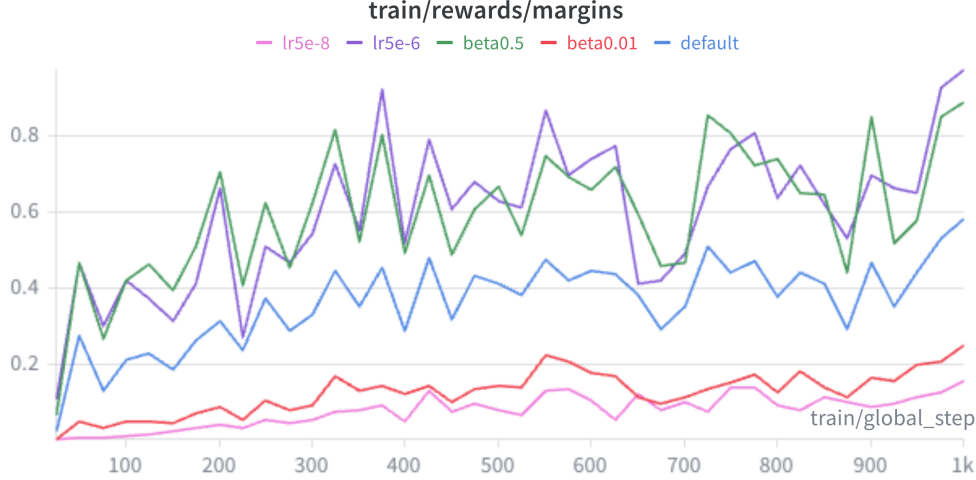


Figure 8: rewards/margins (train), all 5 runs — D (lr5e-6) and C ($\beta=0.5$) sit highest (≈ 0.9 – 1.0 by step 1000), then A (default ≈ 0.58), then B ($\beta=0.01$), with E (lr5e-8) hugging the axis. This is the key 2(c) figure.

Findings

Important correction vs. the naive expectation. The textbook intuition is “smaller $\beta \Rightarrow$ bigger margin” (expected ordering $B > A > C$). Our measured runs show the **opposite**, and the reason is a subtlety in the *definition of the logged metric* that is worth spelling out.

Effect of β (Runs B, A, C). β plays a double role: it is the **KL-leash strength** (small $\beta \Rightarrow$ the policy is allowed to drift further from π_{ref}) *and* it is the **multiplicative scale of the logged reward**, since $\hat{r} = \beta \log \frac{\pi_{\theta}}{\pi_{\text{ref}}}$ and therefore **rewards/margins** $= \beta(\Delta_w - \Delta_l)$ where $\Delta = \log \frac{\pi_{\theta}}{\pi_{\text{ref}}}$.

- **Measured (last-10 mean):** margins(B=0.01) = 0.171 < margins(A=0.1) = 0.437 < margins(C=0.5) = 0.690. So the logged margin is **monotonically increasing in β** — the reverse of the naive ordering.
- **Why:** within only 1000 steps the *raw* log-ratio separation ($\Delta_w - \Delta_l$) does grow larger for smaller β (weaker leash $\Rightarrow$ more drift) — e.g. from A’s margin 0.437 at $\beta = 0.1$ the raw separation is ≈ 4.4 , while B’s 0.171 at $\beta = 0.01$ implies a raw separation ≈ 17 , **4 $\times$ larger drift**, exactly as the “weak penalty drifts more” story predicts. But the logged metric multiplies that separation by β , and the β factor (10 $\times$ smaller for B) **outweighs** the $\sim 4\times$ larger drift, so B’s *displayed* margin is the smallest. For C ($\beta = 0.5$) the leash is tight (smallest raw drift) yet the $\times 0.5$ scale makes the displayed margin the largest.
- **Quality check via rewards/accuracies** (which is *scale-free* — it just counts chosen > rejected): A ≈ 0.665 , B ≈ 0.628 , C ≈ 0.66 are all comparable. So the three runs learn the preference about *equally well in one epoch’s-worth of steps*; what differs across them is mostly the β -scaling of the reward metric, **not** the underlying preference accuracy. This is the key lesson: the numerical size of **rewards/margins** is **not comparable across different β** and is not a direct measure of model quality.

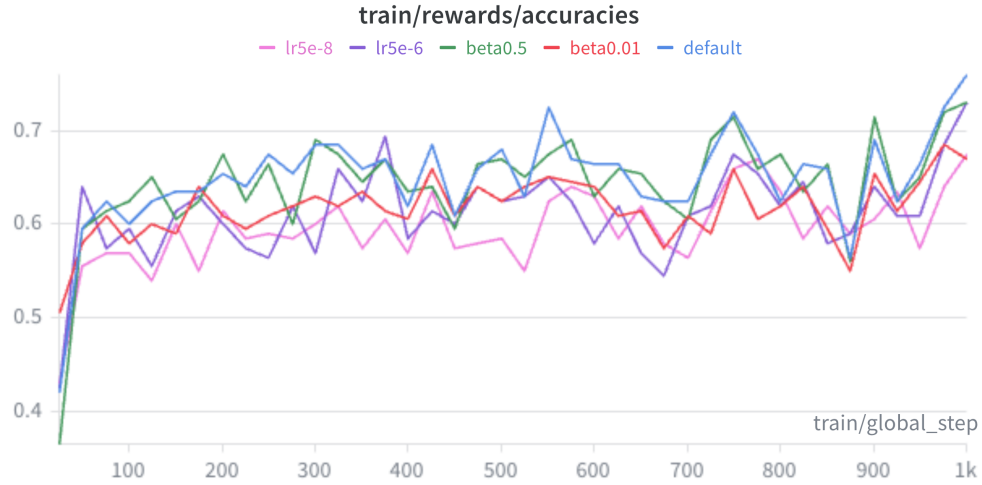


Figure 9: **rewards/accuracies** (train), all 5 runs — all cluster ≈ 0.6 – 0.76 . Preference accuracy is similar across runs even where the margins differ widely, supporting the takeaway that margin size is not a quality score.

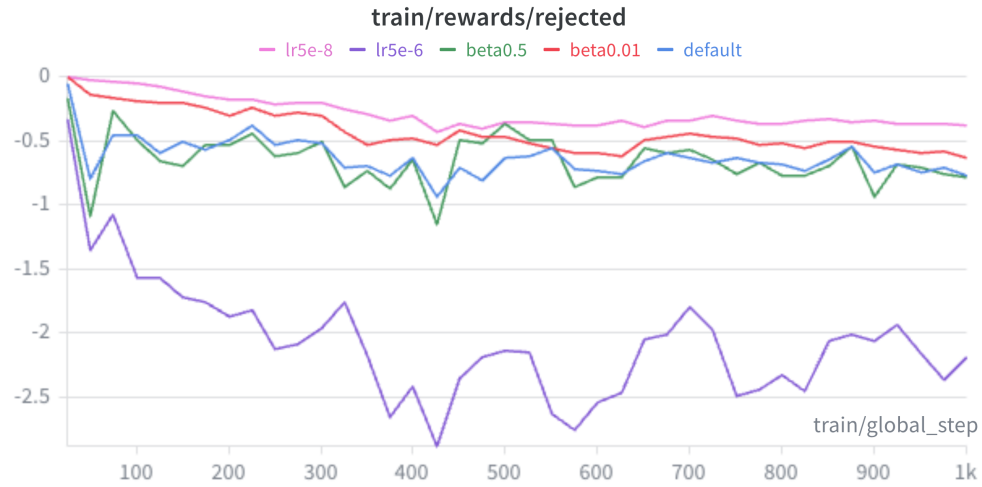


Figure 10: **rewards/rejected** (train), all 5 runs — every run drives the rejected reward below 0; D (high LR) suppresses it most aggressively (≈ -2.5). The growing margin comes mainly from rejected falling.

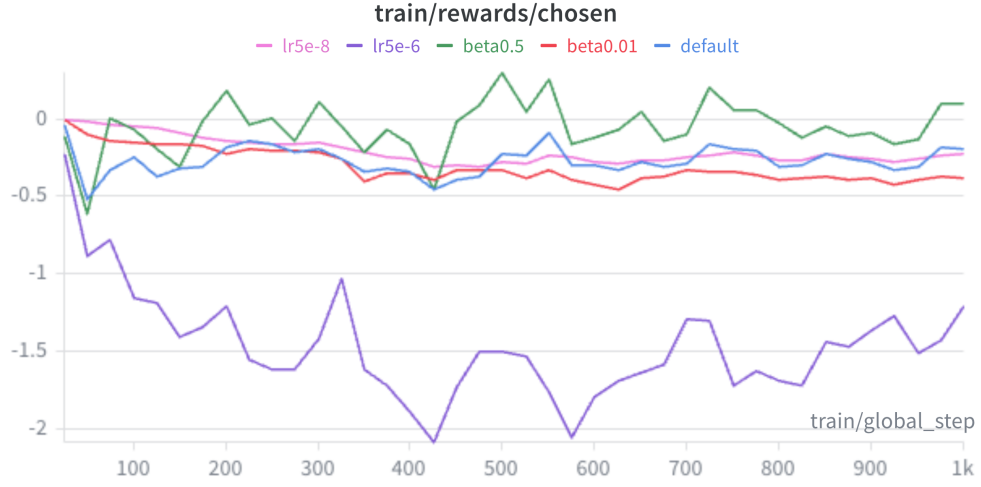


Figure 11: `rewards/chosen` (train), all 5 runs — chosen rewards drift slightly negative for most runs; D pushes chosen down to ≈ -2 , the most of any run.

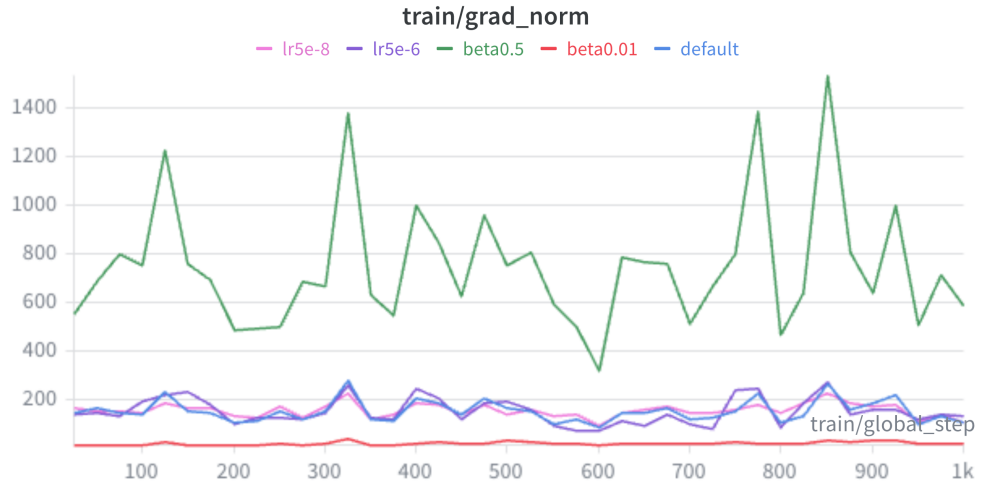


Figure 12: `train/grad_norm`, all 5 runs — gradient norm is governed by β , not LR: C ($\beta=0.5$) is by far the largest and most volatile (≈ 600 – 1500 , $\sim 5\times$ baseline) because the DPO gradient scales with β , while B ($\beta=0.01$) is the smallest. D (high LR) sits right on the baseline cluster (≈ 100 – 250).



Figure 13: **train/loss**, all 5 runs — D (high LR) is the noisiest and highest (spikes to ≈ 0.85 – 0.88); the baseline is the smoothest. LR-induced instability shows up here in the loss, whereas β shows up in grad-norm.

Effect of learning rate (Runs E, A, D). With β fixed at 0.1 the β -scale is **constant**, so here **rewards/margins** is directly comparable across runs and cleanly tracks how fast the policy moves. **Measured (last-10 mean):** $\text{margins}(\text{E}=5\text{e-}8) = 0.111 < \text{margins}(\text{A}=5\text{e-}7) = 0.437 < \text{margins}(\text{D}=5\text{e-}6) = 0.724$, i.e. $\text{margins}(\text{D}) > \text{margins}(\text{A}) > \text{margins}(\text{E})$ — exactly the expected ordering, and the cleanest, most monotonic trend of the whole study (see `plots/margins_compare.png`, where the D curve sits at the top and the E curve hugs the x-axis).

- **Higher LR 5×10^{-6} (Run D):** learns the preference fastest — its margin rises steeply to ≈ 0.72 ($\approx 1.7\times$ the baseline), the largest of A/D/E. As expected the run is **noisier**: in the comparison plots the D curve has the biggest step-to-step swings in **train/loss** (it spikes to ≈ 0.85 – 0.88 , the highest and jumpiest of all five runs) and in **rewards/margins/rewards/chosen** (D pushes **rewards/chosen** down to ≈ -2 , far more than any other run) — the price of the aggressive step size; here it still helps rather than diverging at 1000 steps.
 - *Note on **grad_norm**:* the LR does **not** inflate the gradient norm — **train/grad_norm** for D sits right on top of the baseline cluster (≈ 100 – 250). Gradient norm is instead governed by β (the DPO gradient scales with β): run C ($\beta = 0.5$) has by far the largest, most volatile grad-norms (≈ 600 – 1500 , $\sim 5\times$ baseline) and run B ($\beta = 0.01$) the smallest ($\sim 1/10$). So LR shows up as *loss/margin* noise, while β shows up as *grad_norm* magnitude — two different axes of “noise.”
- **Lower LR 5×10^{-8} (Run E):** updates $\sim 10\times$ smaller than baseline, so in 1000 steps the policy barely moves — margin only ≈ 0.11 and **rewards/accuracies** ≈ 0.62 (just above chance): clear **underfitting**. (It is small but not exactly zero — even tiny steps accumulate a little signal.)
- **Baseline 5×10^{-7} (Run A):** steady, stable growth in between (margin ≈ 0.44 , acc ≈ 0.67).

Note the accuracies: A ≈ 0.665 is actually the **highest** of A/D/E — D’s larger *margin* (0.724) comes with a slightly *lower* accuracy (0.636), a hint that the high LR inflates the reward gap a bit

faster than it improves the actual ranking, the early edge of the speed-vs-stability trade-off.

Summary of measured orderings. - **By β (fixed LR):** margins: $C(0.69) > A(0.44) > B(0.17)$; acc: all ≈ 0.63 – 0.66 . - **By LR (fixed β):** margins: $D(0.72) > A(0.44) > E(0.11)$; acc: $A(0.67) \gtrsim D(0.64) \gtrsim E(0.62)$.

Takeaways. 1. **rewards/margins is not a quality score, and is not comparable across different β .** It is jointly set by the β -scale and by how far the policy has moved. Across A/B/C the scale-free **rewards/accuracies** are nearly equal (≈ 0.63 – 0.66) even though the margins span $0.17 \rightarrow 0.69$ — the spread is almost entirely the β multiplier, *not* better preference learning. When β is held fixed (the LR sweep A/D/E) the margin *does* become a meaningful within-sweep speed indicator. 2. β trades *fidelity to the reference* against *amount of drift* (smaller $\beta \Rightarrow$ larger raw log-ratio drift but a smaller β -scaled displayed margin); LR trades *speed* against *stability* (higher LR $\Rightarrow$ faster, larger, noisier margin). The default ($\beta = 0.1$, LR = 5×10^{-7}) sits in the stable middle of both axes. 3. In every healthy run **rewards/rejected** falls, the margin and **rewards/accuracies** rise, and **loss** decreases — exactly what Problem 1(a)’s gradient predicts (widen the gap, weighting current mistakes). Note **rewards/chosen** need not rise (see the 2(b) note): the objective only needs the *gap* to widen. 4. **The naive “expected” ordering $B > A > C$ was wrong; the measured ordering is $C > A > B$** — and tracing *why* (the dual role of β as both KL-leash and reward-scale) is the main insight of this study. The LR sweep $D > A > E$ matched expectation cleanly because β — and hence the scale — is held fixed there.